

ICS 小班研讨题和作业题（第 6 讲）

注：研讨题的默认前提基础是本课程尤其是本讲的课件和对应教材内容，不一定每次都做强调说明，例如 x86 体系结构、C 语言程序等。

研讨题 1：传递控制和数据

- (1) 基于课件第 4~7 页，自己重写示例，代替课件中的过程 P 和 Q。然后结合新的示例，讲解传递控制、传递数据和内存管理（分配和释放）分别包含哪些内容。
- (2) 课件第 8 页提到的 ABI，主要包含哪些方面的内容？这些内容，哪些在之前课程中讲过，哪些是本次课讲的，哪些还没讲过？
- (3) **拓展研讨：**ABI 包含的主要内容，哪些是由硬件结构决定的（不可修改）；哪些是软件层面的规定（可以修改），但如果修改，会有哪些连带影响？
- (4) 基于 x86-64 体系结构，以 pushq 和 popq 两条指令的执行流程为主线，讲解栈的结构和相关操作。
- (5) 以课件第 19 页的代码为总纲，基于课件第 21~24 页，讲解 call 和 ret 指令的操作流程。按照这样的详细程度：现在执行 xx 指令，位于地址 xx；执行这条指令前，rsp 指向的地址是 xx；执行完这条指令，栈的内容如何变化，rsp 如何变化，下一条指令如何确定；……
- (6) 基于课件第 26 页，讲解 x86-64 上运行 Linux 时，过程参数如何传递。进一步调研，x86-64 和 Windows 的传参规则；32 位 x86（IA32）和 Linux、Windows 的传参规则；简要分析它们之间的差异。
- (7) 讲解课件第 27 页的代码，按照运行顺序，逐行讲解汇编代码，并指出和 C 代码的对应关系。进一步分析：如果 C 代码中的 mult2(x,y)改为 mult2(y,x)，汇编代码会有什么变化。

研讨题 2：栈上的局部存储

- (1) 基于课件第 31 和 43 页的说明，讲解第 32~42 页的变化过程。
- (2) 基于课件第 44~53 页，逐行解读汇编代码的操作，包括对寄存器和栈的影响等。如果汇编代码和 C 代码不对应的（例如，对 rsp 的加减操作，对 32 位寄存器的操作等），要额外解释原因。
- (3) 为什么要区分调用者和被调用者保存的寄存器？参照课件第 54 页，再设计几个不同形式的示例。
- (4) 根据课件第 56 和 57 页，讲解 x86-64 运行 Linux 时的寄存器使用规则。
- (5) **拓展研讨：**x86-64 运行 Windows 时的寄存器使用规则是怎样的，对比和 Linux 上规

则的差异及优缺点。进一步调研并对比 IA32 上运行 Linux 和 Windows 的寄存器使用规则。

(6) 基于课件第 58~65 页，逐行解读汇编代码的操作，包括对寄存器和栈的影响等。如果汇编代码和 C 代码不对应的，要额外解释原因。

(7) 基于课件第 67 页及后续相关内容，逐行解读汇编代码并和 C 代码对应分析。

(8) 有哪些方面的支持（硬件上、软件上），就可以支持递归？为什么还有一些编程语言不支持递归？

作业题

本讲无，合到第 7 讲。